SRI SAI RAM ENGINEERING COLLEGE

SAI LEO NAGAR,WEST TAMBARAM, CHENNAI-44

(An Autonomous Institution)



NAME :

REGISTER NUMBER:

24AMPW501 - BIG DATA ANALYTICS WITH LABORATORY

(III YEAR / V SEM)

(BATCH: 2024–2028)

B.E CSE (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

ACADEMIC YEAR:2026–2027

Sri SAI RAM ENGINEERING COLLEGE
An Autonomous Institution | Affiliated to Anna University & Approved by AICTE, New Delhi
Accredited by NBA and NAAC "A+" | An ISO 9001:2015 Certified and MHRD NIRF ranked institution
Sai Leo Nagar, West Tambaram, Chennai - 600 044. www.sairam.edu.in
Founder Chairman : MJF. Ln. Leo Muthu

Bonafide Certificate

Register No:_____

Certified that this is the Bonafide Record of the work done by Mr./Ms

_____ in the Semester – V of **B.E** Degree course **CSE(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)** in the **24AMPW501 - BIG DATA ANALYTICS WITH LABORATORY**

during the academic year **2026-2027.**

Station:Chennai–600044

Date**:**

STAFF IN-CHARGE **HEAD OF THE DEPARTMENT**

Submitted for University Practical Examination held on _____

at **Sri Sai Ram Engineering College**, Chennai – 600 044.

INTERNAL EXAMINER **EXTERNAL EXAMINAR**

INDEX

S. No	Date	Title of the Experiment	Page No.	Marks	Signature
1		Install Hadoop and configure Hadoop Distributed File System (HDFS) in standalone or pseudo-distributed mode.			
2		Demonstrate basic HDFS commands for file creation, upload, download, replication, and deletion operations.			
3		Develop a MapReduce program to perform Word Count analysis on a text dataset.			
4		Implement Hadoop Streaming using Python scripts for simple data processing tasks.			
5		Perform data storage and retrieval operations using Cassandra Query Language (CQL).			
6		Create and manipulate Key-Value and Document-oriented data models using NoSQL databases.			
7		Develop simple Pig Latin scripts to perform filtering, grouping, joining, and aggregation operations on datasets.			
8		Create Hive tables and execute HiveQL queries including DDL commands and simple analytical queries.			
9		Create data visualizations such as Line Charts, Bar Charts, Pie Charts, Scatter Plots, and Box Plots using Python libraries.			
10		Perform case study implementation involving installation and execution of Hive with sample datasets.			

EXP:01	**INSTALL HADOOP AND CONFIGURE HDFS IN PSEUDO-DISTRIBUTED MODE**

AIM

To install Apache Hadoop 3.5.0 on Windows 11 using WSL2 and Ubuntu, configure HDFS and YARN in pseudo-distributed mode, and verify that the Hadoop daemons run successfully.

REQUIREMENTS

- Windows 11
- WSL2
- Ubuntu
- OpenJDK 17
- Apache Hadoop 3.5.0
- SSH

DESCRIPTION

Hadoop is a distributed data-processing platform. In this setup, Windows 11 hosts WSL2, Ubuntu provides the Linux environment, Java 17 provides the runtime, and Hadoop 3.5.0 provides HDFS and YARN services. Pseudo-distributed mode runs the Hadoop daemons on a single machine while using HDFS and YARN services.

ALGORITHM

1. Enable and verify WSL2 with Ubuntu.
2. Install OpenJDK 17 and verify the Java version.
3. Download and extract Hadoop 3.5.0.
4. Set JAVA_HOME, HADOOP_HOME and related environment variables.
5. Configure hadoop-env.sh.
6. Install and configure SSH.
7. Configure Hadoop XML files.
8. Create NameNode and DataNode storage directories.
9. Format the NameNode.
10. Start HDFS and YARN.
11. Verify the Hadoop daemons using jps.
12. Create and verify a test file in HDFS.

PROGRAM

1. Verify Java and Hadoop

```
echo $JAVA_HOME
echo $HADOOP_HOME
java -version
hadoop version
```

2. Check Hadoop configuration

```
cd $HADOOP_HOME/etc/hadoop
ls
```

3. Format the NameNode

```
hdfs namenode -format
```

4. Start HDFS

```
start-dfs.sh
```

5. Start YARN

```
start-yarn.sh
```

6. Check Hadoop daemons

```
jps
```

7. Create a directory in HDFS

```
hdfs dfs -mkdir -p /user/wwka/input
```

8. Create a sample file

```
echo "Hadoop Distributed File System Test" > test.txt
```

9. Upload the file to HDFS

```
hdfs dfs -put test.txt /user/wwka/input/
```

10. Display the uploaded file

```
hdfs dfs -ls /user/wwka/input

hdfs dfs -cat /user/wwka/input/test.txt
```

OUTPUT

NameNode
DataNode
SecondaryNameNode
ResourceManager
NodeManager
Jps

Hadoop Distributed File System Test

RESULT

Thus, Hadoop was successfully configured in pseudo-distributed mode. HDFS and YARN services were started successfully, and a sample file was created, uploaded, and retrieved from HDFS.

EXP:02	**DEMONSTRATE BASIC HDFS COMMANDS FOR FILE CREATION, UPLOAD, DOWNLOAD, REPLICATION, AND DELETION OPERATIONS.**

AIM

To demonstrate basic Hadoop Distributed File System (HDFS) commands for creating directories, creating files, uploading files, downloading files, checking file information, changing replication, and deleting files.

REQUIREMENTS

- Windows 11
- WSL2
- Ubuntu
- OpenJDK 17
- Apache Hadoop 3.5.0
- HDFS

DESCRIPTION

HDFS is the storage component of Hadoop. It provides commands to create directories and files, upload data from the local file system to HDFS, download data from HDFS, view stored files, change replication factors, and delete files and directories.

ALGORITHM

1. Start HDFS services.
2. Verify the Hadoop daemons using `jps`.
3. Create a directory in HDFS.
4. Create a sample file in the local file system.
5. Upload the file to HDFS.
6. List and verify the files stored in HDFS.
7. Display the contents of the HDFS file.
8. Change the replication factor of the file.
9. Download the file from HDFS to the local file system.
10. Delete the file from HDFS.
11. Delete the HDFS directory.
12. Verify that the files and directories have been deleted.

PROGRAM

1. Start HDFS

```
start-dfs.sh
```

2. Check Hadoop daemons

```
jps
```

3. Create a directory in HDFS

```
hdfs dfs -mkdir -p /user/wwka/exp2
```

4. Create a sample file

```
echo "HDFS Basic Commands Experiment" > exp2.txt
```

5. Upload the file to HDFS

```
hdfs dfs -put exp2.txt /user/wwka/exp2/
```

6. List the HDFS directory

```
hdfs dfs -ls /user/wwka/exp2
```

7. Display the file contents

```
hdfs dfs -cat /user/wwka/exp2/exp2.txt
```

8. Check file information

```
hdfs dfs -stat "%n %b bytes" /user/wwka/exp2/exp2.txt
```

9. Change the replication factor

```
hdfs dfs -setrep -w 1 /user/wwka/exp2/exp2.txt
```

10. Download the file from HDFS

```
hdfs dfs -get /user/wwka/exp2/exp2.txt exp2_downloaded.txt
```

11. Verify the downloaded file

```
cat exp2_downloaded.txt
```

12. Delete the file from HDFS

```
hdfs dfs -rm /user/wwka/exp2/exp2.txt
```

13. Delete the HDFS directory

```
hdfs dfs -rmdir /user/wwka/exp2
```

14. Verify deletion

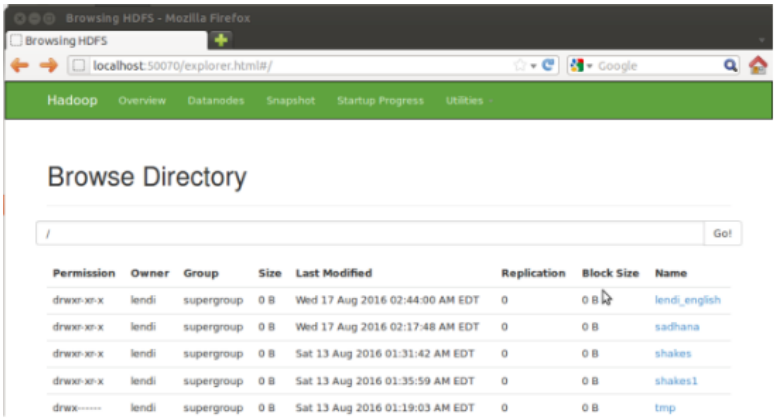
```
hdfs dfs -ls /user/wwka
```

OUTPUT

SAMPLE INPUT:

Input is given as data stored in HDFS using basic HDFS file-management commands.

EXPECTED OUTPUT:



RESULT

Thus, the various file management operations such as file creation, uploading, downloading, replication, listing and deletion were successfully performed using HDFS commands.

EXP:03	**DEVELOP A MAPREDUCE PROGRAM TO PERFORM WORD COUNT ANALYSIS ON A TEXT DATASET**

AIM

To develop a Python MapReduce Word Count program using Hadoop Streaming and count the frequency of each word in a text dataset.

REQUIREMENTS

- Windows 11
- WSL2
- Ubuntu
- OpenJDK 17
- Apache Hadoop 3.5.0
- Python 3

DESCRIPTION

The Mapper reads each input line, extracts words, and emits each word with the value 1. Hadoop performs the Shuffle and Sort phase. The Reducer receives grouped words and sums their counts to produce the final word-frequency table.

ALGORITHM

1. Read each line from standard input in the Mapper.
2. Normalize the line to lowercase and extract words.
3. Emit each word with value 1 as a tab-separated key-value pair.
4. Hadoop groups and sorts identical keys.
5. Read grouped key-value pairs in the Reducer.
6. Add counts for identical words and print the final count.

PROGRAM

1. Create Mapper program

Create the file:

nano mapper.py

```
#!/usr/bin/env python3

import re
import sys

for line in sys.stdin:
    line = line.strip().lower()
```

```
    words = re.findall(r"[a-zA-Z0-9']+", line)

    for word in words:
        print(f"{word}\t1")
```

2. Create Reducer program

nano reducer.py

```
#!/usr/bin/env python3

import sys

current_word = None
current_count = 0

for line in sys.stdin:
    line = line.strip()

    if not line:
        continue

    try:
        word, count = line.split("\t", 1)
        count = int(count)
    except ValueError:
        continue

    if word == current_word:
        current_count += count
    else:
        if current_word is not None:
            print(f"{current_word}\t{current_count}")

        current_word = word
        current_count = count

if current_word is not None:
    print(f"{current_word}\t{current_count}")
```

3. Create input file

nano input.txt

Enter:

```
Hadoop is a Big Data framework
Hadoop uses MapReduce
Python can be used with Hadoop Streaming
MapReduce processes Big Data efficiently
```

4. Test Mapper and Reducer locally

```
cat input.txt | python3 mapper.py | sort | python3 reducer.py
```

5. Create HDFS input directory

```
hdfs dfs -mkdir -p /exp3/input
```

6. Upload input file to HDFS

```
hdfs dfs -put input.txt /exp3/input/
```

7. Execute Hadoop Streaming

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.5.0.jar \

-input /exp3/input \

-output /exp3/output \

-mapper "python3 mapper.py" \

-reducer "python3 reducer.py" \

-file mapper.py \

-file reducer.py
```

8. Verify the output

```
hdfs dfs -cat /exp3/output/part-*
```

OUTPUT

```
a           1
be          1
big         2
can         1
data        2
efficiently 1
framework   1
hadoop      3
is          1
mapreduce   2
processes   1
python      1
streaming   1
used        1
uses        1
with        1
```

RESULT

The Python Word Count MapReduce program was successfully executed using Hadoop Streaming, and the expected word frequencies were obtained.

EXP:04	IMPLEMENT HADOOP STREAMING USING PYTHON SCRIPTS FOR SIMPLE DATA PROCESSING TASKS

AIM

To implement Hadoop Streaming using Python Mapper and Reducer scripts to process student records and calculate the department-wise average marks.

REQUIREMENTS

- Windows 11
- WSL2 with Ubuntu
- OpenJDK 17
- Apache Hadoop 3.5.0
- Python 3

DESCRIPTION

Hadoop Streaming is a utility provided by Apache Hadoop that enables users to create and execute MapReduce applications using scripting languages such as Python. In this experiment, the Mapper processes student records and emits department names and marks. The Reducer calculates the average marks for each department.

ALGORITHM

1. Create a student dataset containing names, departments and marks.
2. Read each student record using the Mapper program.
3. Emit the department as the key and marks as the value.
4. Hadoop performs Shuffle and Sort to group records by department.
5. The Reducer calculates the total marks and number of students in each department.
6. Calculate the average marks for each department.
7. Execute the program using Hadoop Streaming.
8. Display the department-wise average marks.

PROGRAM

1. Create the working directory

```
mkdir -p ~/exp4
cd ~/exp4
```

2. Create the input dataset

```
nano students.txt
```

Enter the following records:

```
Arun,CSE,82
Priya,AIML,94
Karthik,CSE,76
Divya,AIML,88
Meena,CSE,95
Ravi,IT,79
```

3. Create the Mapper program

```
nano mapper.py
```

Enter:

```
#!/usr/bin/env python3

import sys

for line in sys.stdin:
    line = line.strip()

    if not line:
        continue

    parts = line.split(",")

    if len(parts) == 3:
        name, department, marks = parts

        try:
            marks = int(marks)
            print(f"{department}\t{marks}")
        except ValueError:
            continue
```

4. Create the Reducer program

```
nano reducer.py
```

Enter:

```
#!/usr/bin/env python3

import sys

current_department = None
total_marks = 0
student_count = 0

for line in sys.stdin:
```

```
    line = line.strip()

    if not line:
        continue

    department, marks = line.split("\t")
    marks = int(marks)

    if department == current_department:
        total_marks += marks
        student_count += 1
    else:
        if current_department is not None:
            average = total_marks / student_count
            print(f"{current_department}\t{average:.2f}")

        current_department = department
        total_marks = marks
        student_count = 1

if current_department is not None:
    average = total_marks / student_count
    print(f"{current_department}\t{average:.2f}")
```

5. Test the programs locally

```
cat students.txt | python3 mapper.py | sort | python3 reducer.py
```

6. Create the HDFS input directory

```
hdfs dfs -mkdir -p /exp4/input
```

7. Upload the dataset to HDFS

```
hdfs dfs -put students.txt /exp4/input/
```

8. Execute the Hadoop Streaming job

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-
streaming-3.5.0.jar \
-input /exp4/input \
-output /exp4/output \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-file mapper.py \
-file reducer.py
```

9. Display the output

```
hdfs dfs -cat /exp4/output/part-*
```

OUTPUT

```
AIML    91.00
CSE     84.33
IT      79.00
```

RESULT

Thus, Hadoop Streaming was successfully used to process student records with Python Mapper and Reducer scripts and calculate the department-wise average marks.

EXP:05	PERFORM DATA STORAGE AND RETRIEVAL OPERATIONS USING CASSANDRA QUERY LANGUAGE (CQL)

AIM

To perform data storage and retrieval operations using Cassandra Query Language (CQL) by creating a keyspace and table and executing INSERT, SELECT, UPDATE and DELETE operations.

REQUIREMENTS

- Windows 11
- WSL2 with Ubuntu
- Docker Desktop with WSL2 integration
- Apache Cassandra 5.0.9
- Cassandra Query Language Shell (CQLSH)

DESCRIPTION

Apache Cassandra is a distributed NoSQL database designed to handle large amounts of data across multiple nodes. Cassandra Query Language (CQL) is used to create keyspaces and tables and to perform data manipulation operations. In this experiment, student records are stored, retrieved, updated and deleted using CQL.

ALGORITHM

- Start the Cassandra database using Docker.
- Open the Cassandra Query Language Shell.
- Create a keyspace named studentdb.
- Create a table named student.
- Insert student records into the table.
- Retrieve and display the stored records.
- Update the marks of a student.
- Delete a student record.
- Display the final table contents.

PROGRAM

1. Start Cassandra

Run this command in the Ubuntu terminal:

```
docker start cassandra-lab
```

Check its status:

```
docker ps
```

2. Open CQLSH

```
docker exec -it cassandra-lab cqlsh
```

3. Create a keycap

Execute inside CQLSH:

```
CREATE KEYSPACE studentdb

WITH replication = {

   'class': 'SimpleStrategy',

   'replication_factor': 1

};
```

Select the keyspace:

```
USE studentdb;
```

4. Create a student table

```
CREATE TABLE student (
   student_id INT PRIMARY KEY,
   name TEXT,
   department TEXT,
   year INT,
   marks INT
);
```

Verify the table:

```
DESCRIBE student;
```

5. Insert student records

```
INSERT INTO student
(student_id, name, department, year, marks)
VALUES (101, 'Arun', 'CSE', 3, 82);

INSERT INTO student
```

```
(student_id, name, department, year, marks)
VALUES (102, 'Priya', 'AIML', 3, 90);

INSERT INTO student
(student_id, name, department, year, marks)
VALUES (103, 'Karthik', 'CSE', 3, 76);

INSERT INTO student
(student_id, name, department, year, marks)
VALUES (104, 'Divya', 'AIML', 3, 88);

INSERT INTO student
(student_id, name, department, year, marks)
VALUES (105, 'Meena', 'CSE', 3, 95);
```

6. Retrieve all student records

```
SELECT * FROM student;
```

7. Update student marks

Update Priya's marks to 94:

```
UPDATE student
SET marks = 94
WHERE student_id = 102;
```

Verify the update:

```
SELECT * FROM student
WHERE student_id = 102;
```

8. Delete a student record

```
DELETE FROM student
WHERE student_id = 103;
```

9. Display the final records

```
SELECT * FROM student;
```

OUTPUT

student_id	name	department	year	marks
101	Arun	CSE	3	82
102	Priya	AIML	3	94
104	Divya	AIML	3	88
105	Meena	CSE	3	95

RESULT

Thus, data storage and retrieval operations were successfully performed using Cassandra Query Language. Keyspace and table creation, insertion, retrieval, updating and deletion of student records were demonstrated.

EXP:06	CREATE AND MANIPULATE KEY-VALUE AND DOCUMENT-ORIENTED DATA MODELS USING NoSQL DATABASES

AIM

To create and manipulate key-value and document-oriented data models using Python and demonstrate basic CRUD operations: Create, Read, Update and Delete.

REQUIREMENTS

- Windows 11
- WSL2 with Ubuntu
- Python 3
- Python IDLE or terminal

DESCRIPTION

NoSQL databases provide flexible ways to store and manage data without requiring a traditional relational table structure. Key-value data models store information as key-value pairs, while document-oriented models store records as documents containing fields and values.

In this experiment, Python is used to simulate key-value and document-style storage and demonstrate insertion, retrieval, modification and deletion of product records.

ALGORITHM

1. Start the Python program.
2. Create an empty dictionary to store product records.
3. Insert product details using unique product IDs as keys.
4. Display the stored product records.
5. Retrieve a particular product using its ID.
6. Update the price of an existing product.
7. Delete a product record.
8. Insert another product record.
9. Display the final collection of products.
10. Stop the program.

PROGRAM

Save the following code as exp6_nosql.py.

```
products = {}

products["P101"] = {
    "name": "Laptop",
    "category": "Electronics",
    "price": 65000,
    "stock": 20
}

products["P102"] = {
    "name": "Smartphone",
    "category": "Electronics",
    "price": 28000,
    "stock": 35
}

products["P103"] = {
    "name": "Office Chair",
    "category": "Furniture",
    "price": 7500,
    "stock": 15
}

print("INITIAL PRODUCT RECORDS")
for product_id, details in products.items():
    print(product_id, details)
```

```
print("\nRETRIEVING PRODUCT P101")
print(products["P101"])

print("\nUPDATING PRODUCT P102")
products["P102"]["price"] = 26000
print(products["P102"])

print("\nDELETING PRODUCT P103")
del products["P103"]
print("Product P103 deleted successfully")

print("\nINSERTING PRODUCT P104")
products["P104"] = {
    "name": "Tablet",
    "category": "Electronics",
    "price": 22000,
    "stock": 18
}
print(products["P104"])

print("\nFINAL PRODUCT RECORDS")
for product_id, details in products.items():
    print(product_id, details)

print("\nCRUD OPERATIONS COMPLETED SUCCESSFULLY")
```

EXECUTION

Open the Ubuntu terminal and run:

```
python3 exp6_nosql.py
```

OUTPUT

INITIAL PRODUCT RECORDS
P101 {'name': 'Laptop', 'category': 'Electronics', 'price': 65000, 'stock': 20}
P102 {'name': 'Smartphone', 'category': 'Electronics', 'price': 28000, 'stock': 35}
P103 {'name': 'Office Chair', 'category': 'Furniture', 'price': 7500, 'stock': 15}

RETRIEVING PRODUCT P101
{'name': 'Laptop', 'category': 'Electronics', 'price': 65000, 'stock': 20}

UPDATING PRODUCT P102
{'name': 'Smartphone', 'category': 'Electronics', 'price': 26000, 'stock': 35}

DELETING PRODUCT P103
Product P103 deleted successfully

INSERTING PRODUCT P104
{'name': 'Tablet', 'category': 'Electronics', 'price': 22000, 'stock': 18}

FINAL PRODUCT RECORDS
P101 {'name': 'Laptop', 'category': 'Electronics', 'price': 65000, 'stock': 20}
P102 {'name': 'Smartphone', 'category': 'Electronics', 'price': 26000, 'stock': 35}
P104 {'name': 'Tablet', 'category': 'Electronics', 'price': 22000, 'stock': 18}

CRUD OPERATIONS COMPLETED SUCCESSFULLY

RESULT

Thus, key-value and document-style data models were demonstrated using Python dictionaries. The Create, Read, Update and Delete operations were executed successfully.

EXP:07	DEVELOP PIG LATIN SCRIPTS TO PERFORM FILTERING, GROUPING, JOINING AND AGGREGATION

AIM

To develop Pig Latin scripts to perform filtering, grouping, joining and aggregation operations on student datasets using Apache Pig.

REQUIREMENTS

- Windows 11
- WSL2 with Ubuntu
- OpenJDK 17
- Apache Hadoop 3.5.0
- Apache Pig 0.18.0

DESCRIPTION

Apache Pig is a high-level platform used to process large datasets with Hadoop. Pig Latin is its scripting language. It simplifies data processing by providing operations such as LOAD, FILTER, GROUP, JOIN, FOREACH and STORE.

In this experiment, student records are processed to filter students based on marks, calculate department-wise statistics, and join student records with department information.

ALGORITHM

1. Create a student dataset.
2. Create a department dataset.
3. Load the student dataset using Pig Latin.
4. Filter students whose marks are greater than 80.
5. Group students according to their departments.
6. Calculate department-wise student count, average marks and highest marks.
7. Load department information.
8. Join student records with department information.
9. Store and display the results.
10. Stop the execution.

PROGRAM

The following scripts use your existing Apache Pig 0.18.0 installation in WSL2 Ubuntu.

1. Create the working directory

```
mkdir -p ~/exp7_pig/input
cd ~/exp7_pig
```

2. Create the student dataset

```
nano input/students.txt
```

Enter:

```
101,Arun,CSE,82
102,Priya,AIML,94
103,Karthik,CSE,76
104,Divya,AIML,88
105,Meena,CSE,95
106,Ravi,IT,79
```

3. Create the department dataset

```
nano input/departments.txt
```

Enter:

```
CSE,Computer Science
AIML,Artificial Intelligence
IT,Information Technology
```

4. Filtering operation

Create the script:

```
nano filter.pig
```

Program:

```
students = LOAD 'input/students.txt'
USING PigStorage(',')
AS (id:int, name:chararray, dept:chararray, marks:int);

filtered = FILTER students BY marks > 80;

STORE filtered INTO 'output_filter' USING PigStorage(',');
```

EXECUTE

pig -x local filter.pig

Display the output:

cat output_filter/part-m-00000

5. Grouping and aggregation operation

Create the script:

nano summary.pig

Program:

```
students = LOAD 'input/students.txt'
   USING PigStorage(',')
   AS (id:int, name:chararray, dept:chararray, marks:int);

grouped = GROUP students BY dept;

summary = FOREACH grouped GENERATE
   group AS department,
   COUNT(students) AS student_count,
   AVG(students.marks) AS average_marks,
   MAX(students.marks) AS highest_marks;

STORE summary INTO 'output_summary' USING PigStorage(',');
```

Execute:

pig -x local summary.pig

Display the output:

cat output_summary/part-r-00000

6. Joining operation

Create the script:

nano join.pig

Program:

```
students = LOAD 'input/students.txt'
   USING PigStorage(',')
   AS (id:int, name:chararray, dept:chararray, marks:int);

departments = LOAD 'input/departments.txt'
   USING PigStorage(',')
   AS (dept:chararray, dept_name:chararray);

joined = JOIN students BY dept, departments BY dept;

result = FOREACH joined GENERATE
   students::id AS id,
   students::name AS name,
   students::dept AS department,
   departments::dept_name AS department_name,
   students::marks AS marks;

STORE result INTO 'output_join' USING PigStorage(',');
```

Execute:

```
pig -x local join.pig
```

Display the output:

```
cat output_join/part-r-00000
```

OUTPUT

1.

Student ID	Name	Department	Marks
101	Arun	CSE	82
102	Priya	AIML	94
104	Divya	AIML	88
105	Meena	CSE	95

2.

Department	Student Count	Average Marks	Highest Marks
IT	1	79.00	79
CSE	3	84.33	95
AIML	2	91.00	94

3.

Student ID	Name	Department	Department Name	Marks
106	Ravi	IT	Information Technology	79
105	Meena	CSE	Computer Science	95
103	Karthik	CSE	Computer Science	76
101	Arun	CSE	Computer Science	82
104	Divya	AIML	Artificial Intelligence	88
102	Priya	AIML	Artificial Intelligence	94

RESULT

Thus, Pig Latin scripts were successfully used to perform filtering, grouping, aggregation and joining operations on student and department datasets using Apache Pig.

EXP:08	CREATE HIVE TABLES AND EXECUTE HIVEQL QUERIES

AIM

To create Hive tables and execute HiveQL commands, including DDL operations and analytical queries, on a student dataset.

REQUIREMENTS

- Java 17
- Hadoop 3.5.0
- Apache Hive 4.1.0
- Student dataset
- Windows 11 with WSL2 Ubuntu

DESCRIPTION

Apache Hive is a data warehouse system built on Hadoop. It uses HiveQL, a SQL-like language, to create tables, store structured data, and perform queries such as filtering, sorting, grouping, and aggregation.

ALGORITHM

1. Start Hadoop services.
2. Start Hive and connect using Beeline.
3. Create a database for student records.
4. Create a student table.
5. Insert student records.
6. Display all student records.
7. Execute filtering and aggregation queries.
8. Display the query results.
9. Stop the services.

PROGRAM

1. Start Hadoop and open Beeline

Run in Ubuntu terminal:

```
start-dfs.sh
start-yarn.sh
beeline -u jdbc:hive2://localhost:10000
```

2. Create the database and table

Execute in Beeline:

```
CREATE DATABASE IF NOT EXISTS studentdb;
USE studentdb;

CREATE TABLE IF NOT EXISTS student (
   student_id INT,
   name STRING,
   department STRING,
   marks INT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

3. Insert student records:

```
INSERT INTO student VALUES
(101, 'Arun', 'CSE', 82),
(102, 'Priya', 'AIML', 94),
(103, 'Karthik', 'CSE', 76),
(104, 'Divya', 'AIML', 88),
(105, 'Meena', 'CSE', 95),
(106, 'Ravi', 'IT', 79);
```

4. Retrieve all records

```
SELECT * FROM student;
```

5. Filter students scoring above 80

```
SELECT student_id, name, department, marks
FROM student
WHERE marks > 80;
```

6. Calculate department-wise average marks

```
SELECT department,
     COUNT(*) AS student_count,
     ROUND(AVG(marks), 2) AS average_marks,
     MAX(marks) AS highest_marks
FROM student
GROUP BY department;
```

OUTPUT

1. Database and Table Creation

Command	Expected result
`CREATE DATABASE studentdb`	Database created
`USE studentdb`	Database selected
`CREATE TABLE student`	Table created

2. Student Records

Student ID	Name	Department	Marks
101	Arun	CSE	82
102	Priya	AIML	94
103	Karthik	CSE	76
104	Divya	AIML	88
105	Meena	CSE	95
106	Ravi	IT	79

3. Filtering Output — Marks Above 80

Student ID	Name	Department	Marks
101	Arun	CSE	82
102	Priya	AIML	94
104	Divya	AIML	88
105	Meena	CSE	95

4. Department-wise Aggregation Output

Department	Student Count	Average Marks	Highest Marks
CSE	3	84.33	95
AIML	2	91.00	94
IT	1	79.00	79

RESULT

Thus, HiveQL commands were prepared to create a database and table, retrieve student records, filter students by marks, and calculate department-wise statistics.

EXP:09	CREATE DATA VISUALIZATIONS USING PYTHON

AIM

To create data visualizations such as line charts, bar charts, pie charts, scatter plots, and box plots using Python libraries.

REQUIREMENTS

- Windows 11 with WSL2 Ubuntu or Python environment
- Python 3
- Pandas library
- Matplotlib library

DESCRIPTION

Data visualization is the graphical representation of data. Python libraries such as Pandas and Matplotlib help represent data using different types of charts to understand patterns, comparisons, distributions, and relationships.

ALGORITHM

1. Import Pandas and Matplotlib libraries.
2. Create a dataset containing monthly sales and profit values.
3. Convert the dataset into a DataFrame.
4. Create a line chart for monthly sales.
5. Create a bar chart to compare monthly sales.
6. Create a pie chart to show sales distribution.
7. Create a scatter plot to compare sales and profit.
8. Create a box plot to display sales distribution.
9. Display all five visualizations.

PROGRAM

1. Install the required libraries

```
python3 -m pip install pandas matplotlib
```

2. Create the Python program

```
import pandas as pd

import matplotlib.pyplot as plt

data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May"],
    "Sales": [120, 150, 180, 160, 210],
    "Profit": [20, 30, 42, 35, 50]
}

df = pd.DataFrame(data)

print("SALES DATA")
print(df)

plt.figure()
plt.plot(df["Month"], df["Sales"], marker="o")
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid(True)
plt.show()

plt.figure()
plt.bar(df["Month"], df["Sales"])
plt.title("Sales by Month")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.figure()
plt.pie(
    df["Sales"],
    labels=df["Month"],
    autopct="%1.1f%%"
)
plt.title("Monthly Sales Distribution")
plt.show()
```

```
plt.figure()
plt.scatter(df["Sales"], df["Profit"])
plt.title("Sales vs Profit")
plt.xlabel("Sales")
plt.ylabel("Profit")
plt.grid(True)
plt.show()

plt.figure()
plt.boxplot(df["Sales"])
plt.title("Sales Distribution")
plt.ylabel("Sales")
plt.show()

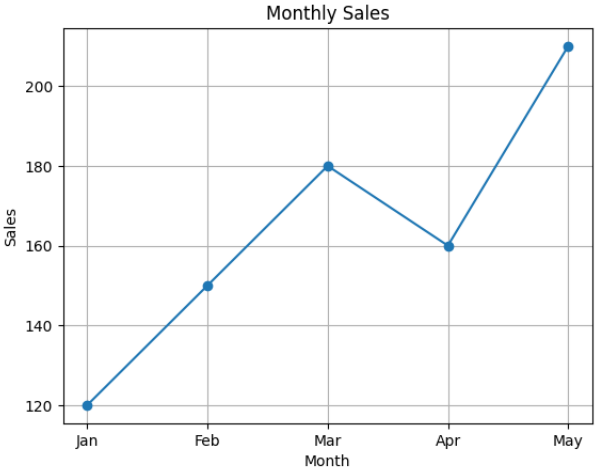
print("All five visualizations created successfully.")
```

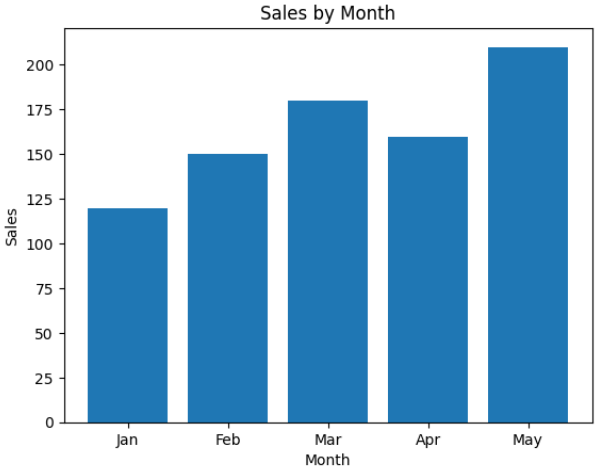
3. Execute the program

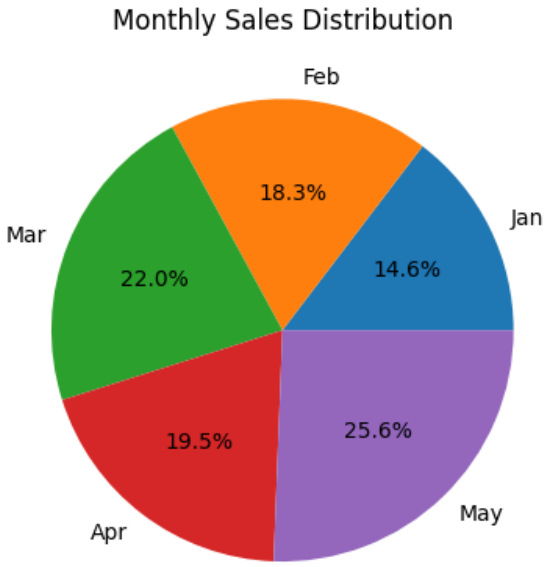
```
python3 visualization.py
```

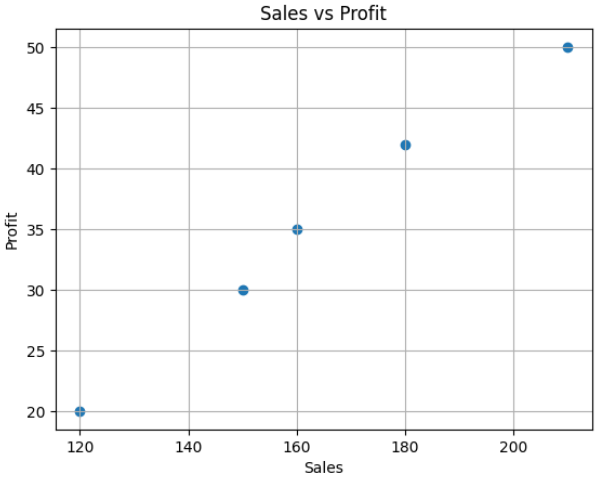
OUTPUT

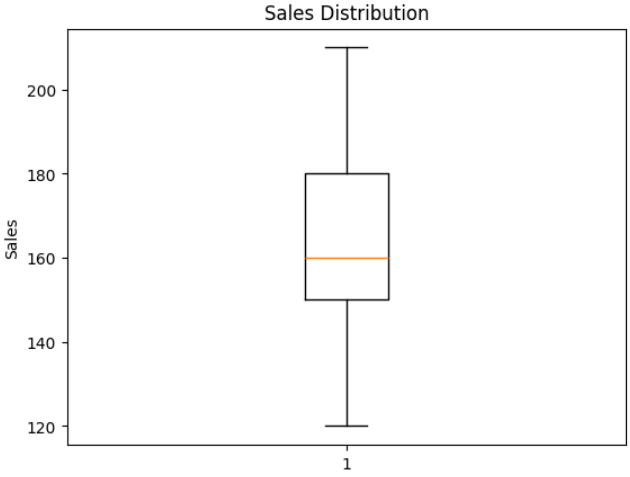
```
SALES DATA
  Month  Sales  Profit
0   Jan    120      20
1   Feb    150      30
2   Mar    180      42
3   Apr    160      35
4   May    210      50
```











RESULT

Thus, five types of data visualizations—line chart, bar chart, pie chart, scatter plot, and box plot—were created using Python, Pandas, and Matplotlib.

EXP:10	CASE STUDY IMPLEMENTATION OF HIVE WITH SAMPLE DATASETS

AIM

To implement a case study using Apache Hive by creating tables, loading sample datasets, and executing HiveQL queries for data analysis.

REQUIREMENTS

- Windows 11 with WSL2 Ubuntu
- Java 17
- Hadoop 3.5.0
- Apache Hive 4.1.0
- Sample student dataset

DESCRIPTION

Apache Hive is a data warehouse framework that runs on Hadoop and supports querying structured data using HiveQL. In this case study, student records are used to demonstrate table creation, data loading, filtering, and department-wise aggregation.

ALGORITHM

1. Start Hadoop HDFS and YARN services.
2. Start Hive and connect using Beeline.
3. Create a database named `studentdb`.
4. Create a table to store student details.
5. Load sample student records into the table.
6. Retrieve all student records.
7. Filter students based on their marks.
8. Calculate department-wise student count and average marks.
9. Display the query results.
10. Stop the services.

PROGRAM

1. Start Hadoop services

Run in the Ubuntu terminal:

```
start-dfs.sh
start-yarn.sh
```

Connect to Hive using Beeline:

```
beeline -u jdbc:hive2://localhost:10000
```

2. Create database and table

Execute the following commands in Beeline:

```
CREATE DATABASE IF NOT EXISTS studentdb;
USE studentdb;

CREATE TABLE IF NOT EXISTS student_case (
   student_id INT,
   name STRING,
   department STRING,
   marks INT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

3. Create the sample dataset

Exit Beeline using `!quit`, then run these commands in Ubuntu:

```
mkdir -p ~/exp10

cd ~/exp10

cat > students.csv << 'EOF'
101,Arun,CSE,82
102,Priya,AIML,94
103,Karthik,CSE,76
104,Divya,AIML,88
105,Meena,CSE,95
106,Ravi,IT,79
EOF
```

Upload the dataset to HDFS:

```
hdfs dfs -mkdir -p /exp10/input
hdfs dfs -put -f students.csv /exp10/input/
```

4. Load data into the Hive table

Reconnect using Beeline:

beeline -u jdbc:hive2://localhost:10000

Execute:

```
USE studentdb;

LOAD DATA INPATH '/exp10/input/students.csv'

INTO TABLE student_case;
```

5. Retrieve all records

```
SELECT * FROM student_case;
```

6. Filter students scoring above 80

```
SELECT student_id, name, department, marks
FROM student_case
WHERE marks > 80;
```

7. Perform department-wise analysis

```
SELECT department,
    COUNT(*) AS student_count,
    ROUND(AVG(marks), 2) AS average_marks,
    MAX(marks) AS highest_marks
FROM student_case
GROUP BY department;
```

OUTPUT

1. Student Records

Student ID	Name	Department	Marks
101	Arun	CSE	82
102	Priya	AIML	94
103	Karthik	CSE	76
104	Divya	AIML	88
105	Meena	CSE	95
106	Ravi	IT	79

2. Filtering Output — Marks Above 80

Student ID	Name	Department	Marks
101	Arun	CSE	82
102	Priya	AIML	94
104	Divya	AIML	88
105	Meena	CSE	95

3. Department-wise Analysis

Department	Student Count	Average Marks	Highest Marks
CSE	3	84.33	95
AIML	2	91.00	94
IT	1	79.00	79

RESULT

Thus, a case study using Apache Hive was prepared to demonstrate database and table creation, loading a sample dataset, filtering student records, and performing department-wise analytical queries.